

Java Singleton Design Pattern

Practical SDET Framework Example • Pros & Cons • Interview Questions • Interactive Revision

1. What is Singleton?

Definition: Singleton is a design pattern that ensures a class has **only one instance** and provides a common way to access that instance.

Think of it as: one class → one shared object → one controlled access point.

SDET analogy: A framework-wide configuration object can be shared when the configuration is read-only and has one logical lifecycle.

2. Basic Singleton Implementation

```
public class ConfigurationManager {

    // Store the single instance.
    private static ConfigurationManager instance;

    // Prevent external object creation.
    private ConfigurationManager() {
    }

    // Return the same instance every time.
    public static ConfigurationManager getInstance() {

        if (instance == null) {
            instance = new ConfigurationManager();
        }

        return instance;
    }
}
```

Usage:

```
ConfigurationManager config1 =
    ConfigurationManager.getInstance();

ConfigurationManager config2 =
    ConfigurationManager.getInstance();

// Both references point to the same object.
System.out.println(config1 == config2); // true
```

Interview point: private constructor + static instance + public static accessor are the core building blocks.

3. Practical SDET Example — Configuration Manager

Suppose your automation framework needs environment, base URL, browser and timeout. A single read-only configuration object can prevent repeated object creation and keep configuration access centralised.

```
public class ConfigManager {

    private static ConfigManager instance;

    private final String environment;
    private final String baseUrl;
    private final int timeout;

    private ConfigManager() {

        // In a real framework these could come
        // from a properties/YAML configuration file.
        environment = "QA";
    }
}
```

```

baseUrl = "https://qa.example.com";
timeout = 30;
}

public static ConfigManager getInstance() {

    if (instance == null) {
        instance = new ConfigManager();
    }

    return instance;
}

public String getEnvironment() {
    return environment;
}

public String getBaseUrl() {
    return baseUrl;
}

public int getTimeout() {
    return timeout;
}
}

```

Page Object usage:

```

public class LoginPage {

    public void openLoginPage() {

        ConfigManager config =
        ConfigManager.getInstance();

        String url = config.getBaseUrl();

        System.out.println("Opening: " + url);
    }
}

```

Another framework class gets the same instance:

```

ConfigManager config1 =
ConfigManager.getInstance();

ConfigManager config2 =
ConfigManager.getInstance();

System.out.println(config1 == config2);
// true

```

4. Pros of Singleton

Advantage	Practical benefit
Single instance	Controls creation of a shared object.
Centralised access	All consumers use one controlled access point.
Resource control	Useful when a resource/service has one logical lifecycle.
Consistent read-only state	A shared immutable configuration can provide consistent values.
Convenient access	Consumers can retrieve the service without passing it through every method.

5. Cons of Singleton

Disadvantage	Practical risk
--------------	----------------

Global/shared state	A mutable Singleton can behave like global state.
Harder testing	Direct getInstance() calls can make mocking/replacement harder.
Parallel execution risk	Mutable state can be overwritten by another test/thread.
Hidden dependencies	A class can depend on the Singleton without declaring that dependency explicitly.
Lifecycle complexity	Shared objects may require careful reset/cleanup.

6. Critical SDET Point — Singleton + Parallel Testing

A Singleton is **not automatically thread-safe**. The classic lazy implementation can allow two threads to create two instances during concurrent first access.

```
// ■ Basic lazy Singleton is not thread-safe.

if (instance == null) {
    instance = new ConfigManager();
}
```

More importantly, even a thread-safe Singleton can still be a **bad design** if it stores mutable per-test data.

```
// Two parallel tests share the same object.

Thread-1 → config.setTestData("User-A");
Thread-2 → config.setTestData("User-B");

// Thread-1 can unexpectedly observe User-B.
// Shared mutable state causes test interference.
```

Selenium warning: Don't use one global Singleton WebDriver for parallel tests. Prefer ThreadLocal<WebDriver> or a per-test driver lifecycle.

```
Thread-1 → Driver-1
Thread-2 → Driver-2
Thread-3 → Driver-3
```

7. Thread-Safe Singleton — Interview Example

```
public class ConfigManager {

    private static ConfigManager instance;

    private ConfigManager() {
    }

    // Synchronised accessor prevents concurrent
    // creation of multiple instances.
    public static synchronized ConfigManager getInstance() {

        if (instance == null) {
            instance = new ConfigManager();
        }

        return instance;
    }
}
```

For interviews, mention that other options include eager initialisation, the holder idiom, double-checked locking, and enum Singleton. The appropriate choice depends on lifecycle, concurrency and testability requirements.

8. Eager vs Lazy Singleton

```
// Eager initialisation.
// Instance is created during class initialisation.

private static final ConfigManager INSTANCE =
    new ConfigManager();
```

```

public static ConfigManager getInstance() {
    return INSTANCE;
}

// Lazy initialisation.
// Instance is created only when first requested.

if (instance == null) {
    instance = new ConfigManager();
}

```

Interview distinction: Eager is simple and safe during class initialisation; lazy delays creation but requires concurrency consideration.

9. Singleton vs Static Utility Class

```

// Static utility class

public final class StringUtils {

    private StringUtils() {
        // Prevent object creation.
    }

    public static boolean isEmpty(String value) {
        return value == null || value.isEmpty();
    }
}

```

A Singleton represents one actual object instance. A utility class generally prevents instantiation and exposes static operations. They should not be treated as interchangeable patterns.

10. Senior SDET Interview Questions

Q1. What problem does Singleton solve?

It controls object creation so one shared instance is available through a common access point.

Q2. What are the essential parts?

Private constructor, static instance field, and static access method.

Q3. Is basic lazy Singleton thread-safe?

No. Concurrent first access can create multiple instances.

Q4. Should WebDriver be a Singleton in parallel Selenium?

No. A shared driver can cause cross-thread interference. Use ThreadLocal or per-test driver lifecycle.

Q5. Where can Singleton fit in an automation framework?

Potentially immutable configuration or carefully designed central services; avoid it for mutable test-specific state.

Q6. Biggest risk?

Shared mutable state and hidden dependencies, especially during parallel execution.

Q7. Singleton or Dependency Injection?

Dependency injection makes dependencies more explicit and generally improves testability. Use Singleton only when one shared lifecycle is genuinely required.

Q8. What makes a Singleton senior-level discussion?

Concurrency, mutability, lifecycle, testability and parallel execution—not just the private constructor.

11. Practical Interview Scenario

Interviewer: “Our framework has a Singleton WebDriver. TestNG runs 10 tests in parallel. What can go wrong?”

Answer: Multiple threads can act on the same browser session. One test may navigate, click or close the browser while another test is using it. This creates race conditions and flaky/interdependent tests. I would use ThreadLocal<WebDriver>

or a per-test driver lifecycle and keep shared Singleton state limited to resources that genuinely have one lifecycle.

12. Quick Revision

SINGLETON

↓

One controlled instance

Core

- private constructor
- static instance
- getInstance()

Pros

- centralised access
- controlled creation
- useful shared lifecycle
- consistent immutable state

Cons

- global/shared state
- harder mocking
- hidden dependencies
- parallel-test interference
- lifecycle/reset concerns

SDET RULE

- Do not blindly make WebDriver a Singleton.
- Prefer ThreadLocal<WebDriver> for parallel execution.
- Keep shared state immutable where possible.

■ **13-year SDET takeaway:** “I use Singleton selectively. I first ask whether the object genuinely needs one shared lifecycle, whether its state is mutable, how it behaves under concurrency, and whether the design remains testable.”